# Overview: Ray Cluster Initialization

## Executive Summary

The `init_ray_cluster_working.py` module provides a reusable function for initializing Ray with safe defaults for distributed computing. Ray is used throughout the pipeline for parallel processing, especially in OER metrics computation and pool metrics calculations.

**Core Concept**: Initialize Ray with appropriate resource allocation (CPUs, object store memory) and configuration for the analysis environment.

---

## Function Overview

`init_ray_cluster(cpus, obj_store_gb, temp_dir, dis_mem_mon)`

**Purpose**: Cleanly initialize Ray with safe defaults for systems with 16 CPUs and ~124 GiB RAM.

**Parameters**: - `cpus` (int): Number of CPUs Ray can use (default: 12) - `obj_store_gb` (int): Shared memory size for object store in GiB (default: 40) - `temp_dir` (str): Temporary directory for Ray spillover (default: "/tmp/ray_tmp") - `dis_mem_mon` (str): Memory monitor setting - "0" = enabled, "1" = disabled (default: "0")

**Returns**: None (initializes Ray cluster)

**Usage**:

```
from init_ray_cluster_working import init_ray_cluster

# Initialize with defaults (12 CPUs, 40 GB object store)
init_ray_cluster()

# Or customize
init_ray_cluster(cpus=16, obj_store_gb=40, temp_dir="/tmp/ray_tmp", dis_mem_mon="0")
```

---

## Configuration Details

### Environment Variables Set

1. **MODIN_ENGINE**: Set to "ray" to enable Modin's Ray backend
2. **RAY_DISABLE_MEMORY_MONITOR**: Controls memory monitoring ("0" = enabled, "1" = disabled)
3. **RAY_TEMPDIR**: Temporary directory for Ray spillover

### Ray Initialization

The function calls `ray.init()` with: - `num_cpus`: Number of CPUs to use - `object_store_memory`: Object store size in bytes (converted from GiB) - `ignore_reinit_error`: Allows re-initialization if Ray is already running

### Memory Monitor

The memory monitor helps prevent out-of-memory (OOM) errors by: - Monitoring worker memory usage - Killing workers before the system runs out of memory - Recommended: Keep enabled (`dis_mem_mon="0"`) - Use `dis_mem_mon="1"` only for debugging if you need to disable monitoring

---

## Integration with Pipeline

### Usage in 520 Pipeline

The function is imported in CELL 0 but typically not called directly (commented out):

```
from init_ray_cluster_working import init_ray_cluster
# init_ray_cluster()  # Usually commented out
```

Ray is initialized automatically by: - Modin when it detects Ray backend - `add_cumulative_oer_metrics()` when it uses `@ray.remote`

### Default Configuration

The pipeline typically uses: - **CPUs**: 16 (all available cores) - **Object Store**: 40 GB (for large lookup dictionaries) - **Temp Directory**: `/tmp/ray_tmp` - **Memory Monitor**: Enabled

---

## Best Practices

### 1. Object Store Size

Set `obj_store_gb` based on: - Size of lookup dictionaries (typically 10-20 GB) - Number of parallel tasks - Available system memory

**Recommendation**: 40 GB for datasets with 100,000+ officers

### 2. CPU Allocation

Set `cpus` to: - Number of available CPU cores - Leave 1-2 cores for system processes

**Recommendation**: 16 CPUs for systems with 16+ cores

### 3. Memory Monitor

Keep memory monitor enabled (`dis_mem_mon="0"`) to: - Prevent OOM errors - Automatically kill workers before system runs out of memory

### 4. Temp Directory

Use a dedicated temp directory: - Separate from system temp - Large enough for spillover (typically 10-20 GB) - Regularly cleaned (see `ray_tmp_clean_working.sh`)

---

## Troubleshooting

### Common Issues

1. **Ray Already Initialized**: Use `ignore_reinit_error=True` (already set in function)
2. **OOM Errors**: Increase `obj_store_gb` or reduce number of parallel tasks
3. **Temp Directory Full**: Clean `/tmp/ray_tmp` using `ray_tmp_clean_working.sh`
4. **Low CPU Utilization**: Check that Ray is using all available CPUs

### Debugging Tips

1. **Check Ray Status**: Use `ray.status()` to see current configuration
2. **Monitor Memory**: Check system memory usage during computation
3. **Review Logs**: Check Ray logs for errors or warnings

---

## Related Documentation

- **520_PIPELINE_COX_OVERVIEW.md**: Overall pipeline documentation
- **ADD_CUM_OER_METRICS_OVERVIEW.md**: OER metrics computation (uses Ray for parallelization)

---

*Last Updated: January 22, 2026*